

DELIVERABLE 3: Lean PRD (Product Requirements Document)

Problem & Users

Target Users:

- Primary: Technical professionals (engineers, data scientists, researchers)
- Secondary: Students, analysts, consultants

Core Problem: Information retrieval is slow, imprecise, and lacks contextual understanding. Users need instant, accurate answers from large document collections with proper citations.

Goals & Success Metrics

Product Goals:

1. Return accurate answers in <3 seconds
2. Achieve >85% answer relevance (human evaluation)
3. Provide source citations for every claim
4. Handle 100+ concurrent users
5. Cost <\$0.05 per query

Success Metrics:

- **Technical:** Query latency (p95 <3s), retrieval precision @k, citation accuracy
- **Business:** User satisfaction score >4/5, query success rate >90%
- **Operational:** System uptime >99.5%, cost per 1K queries <\$5

Core Features

MVP Features:

1. **Document Ingestion Pipeline**
 - PDF/Markdown support
 - Automatic chunking and embedding
 - Metadata extraction
2. **Hybrid Retrieval System**
 - Vector similarity search (semantic)
 - Keyword search (BM25)
 - Reranking for precision
3. **Fine-Tuned Response Generation**
 - Domain-adapted LLM (7B parameters)

- Citation injection
- Hallucination guardrails
- 4. **Web API & UI**
 - RESTful API endpoints
 - Simple React chat interface
 - Query history
- 5. **Monitoring Dashboard**
 - Query analytics
 - Latency tracking
 - Cost monitoring

Non-Goals

- **✗** Multi-modal inputs (images, audio) - Keep scope manageable
- **✗** Multi-language support - English only for MVP
- **✗** Real-time document updates - Batch ingestion sufficient
- **✗** Advanced authentication - Simple API keys adequate
- **✗** Mobile apps - Web-first approach

Risks & Constraints

Technical Risks:

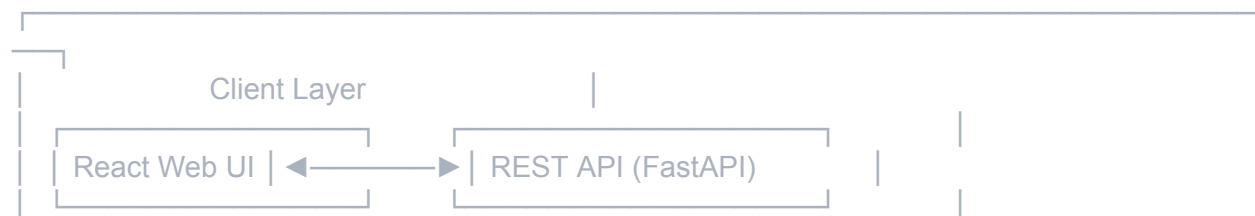
- LLM hallucinations despite RAG → Mitigation: Citation requirements, confidence scoring
- High inference latency → Mitigation: Model quantization, caching, async processing
- Vector DB scaling → Mitigation: Start with FAISS, migration path to managed service

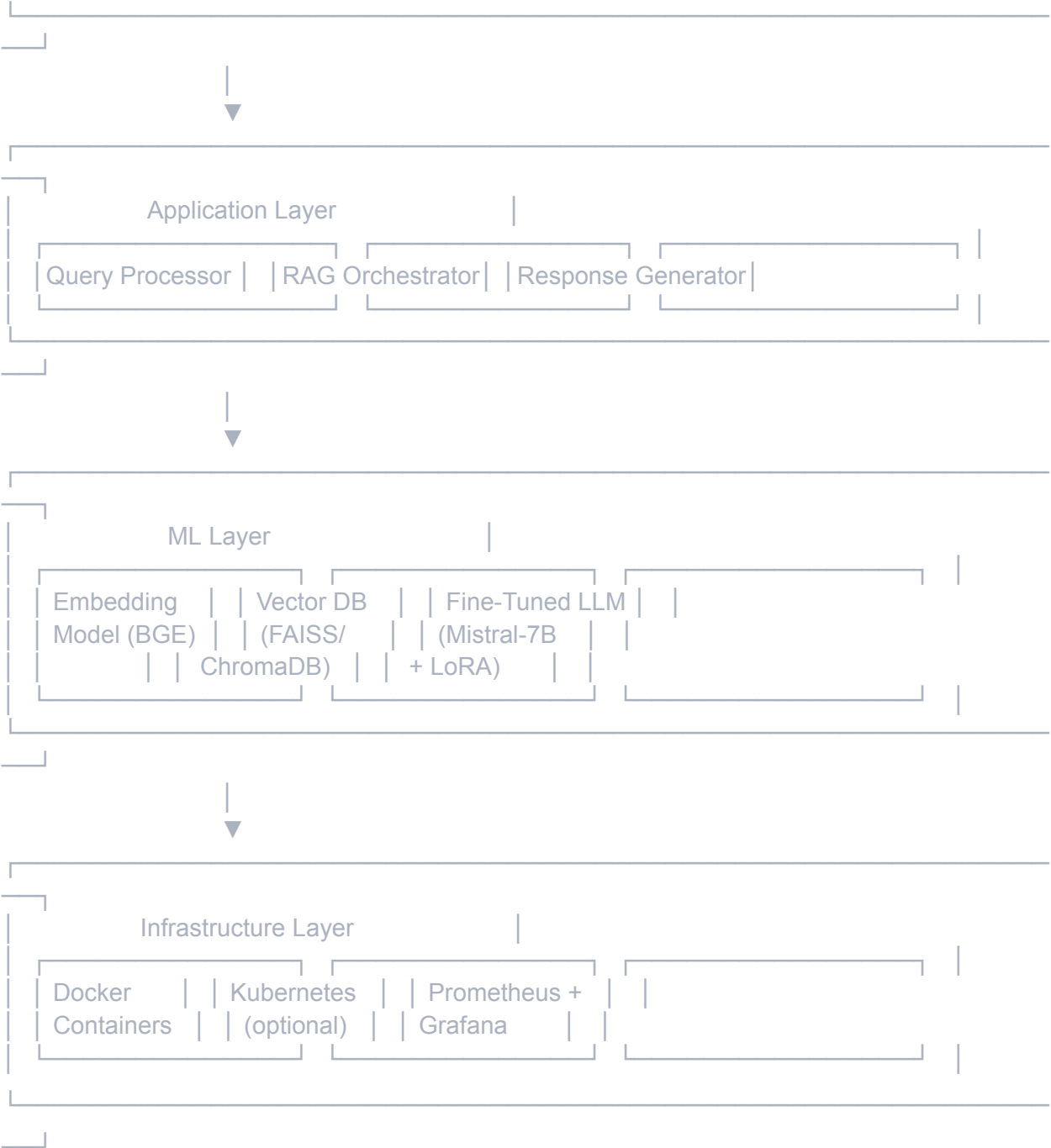
Resource Constraints:

- Limited compute budget → Use smaller models (7B), quantization, spot instances
- Time constraint (8-12 weeks) → MVP-first, iterative releases
- Single developer → Prioritize automation, use managed services where possible

DELIVERABLE 4: Technical Design Document

System Architecture





Data Sources & Pipelines

Document Ingestion Pipeline:

- 1. **Input Sources:**
 - Research papers (ArXiv API)
 - Technical documentation (GitHub repos)

- Custom PDFs (user uploads)

2. Processing Steps:

Raw Documents → PDF Parser → Text Extraction →
Chunking (512 tokens, 50 overlap) → Embedding Generation →
Vector DB Storage + Metadata Indexing

3. Technologies:

- PDF Parsing: PyMuPDF, pdfplumber
- Chunking: LangChain RecursiveCharacterTextSplitter
- Embeddings: BGE-base-en-v1.5 (best quality/latency tradeoff)
- Storage: ChromaDB (development), Pinecone (production option)

Model Selection & Training

Embedding Model:

- **Choice:** BAAI/bge-base-en-v1.5
- **Rationale:** Fine-tuned compact models like bge-base-en-v1.5 achieve superior trade-off between accuracy, latency, and cost compared to larger models [arXiv](#)
- **Fine-tuning:** Domain-specific embedding adaptation on target corpus

LLM Selection:

- **Choice:** Mistral-7B-Instruct-v0.2
- **Rationale:**
 - Open-source, commercially viable
 - Excellent instruction-following
 - 32K context window (handles long documents)
 - Efficient inference (vs 70B models)

Fine-Tuning Strategy:

- **Method:** QLoRA (4-bit quantization + LoRA adapters)
- **Training Data:**
 - Synthetic Q&A pairs generated from corpus
 - Human-annotated examples (high-quality subset)
 - Emphasis on citation accuracy
- **Hyperparameters:**

python

LoRA_r = 16

LoRA_alpha = 32

Batch_size = 4 (gradient accumulation)

Learning_rate = 2e-4

Epochs = 3-5 (early stopping on validation loss)

Evaluation Metrics

Retrieval Quality:

- Precision@k, Recall@k (k=3,5,10)
- Mean Reciprocal Rank (MRR)
- Normalized Discounted Cumulative Gain (NDCG)

Generation Quality:

- ROUGE-L (overlap with reference answers)
- BERTScore (semantic similarity)
- Human evaluation rubric:
 - Accuracy (1-5)
 - Relevance (1-5)
 - Citation correctness (binary)

System Performance:

- Query latency (p50, p95, p99)
- Throughput (queries/second)
- Cost per query
- System uptime

Deployment Strategy

Phase 1: Local Development

- Docker Compose setup
- Local FAISS vector store
- Gradio/Streamlit UI for quick iteration

Phase 2: Cloud Deployment (AWS)

...

Infrastructure:

- API Server: EC2 t3.medium (or Fargate containers)
- Model Serving: EC2 g4dn.xlarge (1x T4 GPU) or SageMaker
- Vector DB: Self-hosted ChromaDB or Pinecone managed
- Load Balancer: AWS ALB
- Monitoring: CloudWatch + Prometheus/Grafana
- Storage: S3 for documents, EFS for model weights

...

Phase 3: Kubernetes (Optional Stretch)

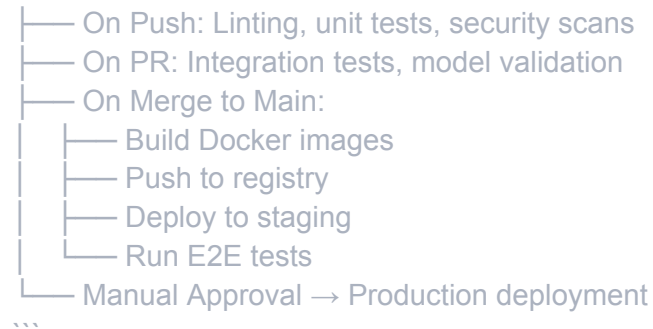
- Helm charts for deployment

- Horizontal Pod Autoscaling (HPA) based on CPU/GPU
- KServe **for** model serving

CI/CD Pipeline:

...

GitHub Actions:



Scalability & Monitoring

Scalability Considerations:

1. **Horizontal Scaling:**

- Stateless API servers behind load balancer
- Model serving **with** batching (increase throughput)
- Vector DB sharding **for** large document collections

2. **Performance Optimizations:**

- Response caching (Redis) **for** common queries
- Model quantization (int8/int4)
- Prompt caching (**reduce** redundant processing)
- Async processing **for** non-critical paths

3. **Cost Optimization:**

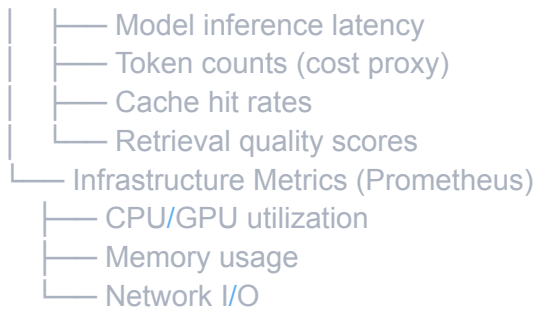
- Use spot instances **for** batch ingestion
- Auto-scaling policies (scale to zero during idle)
- Monitor token usage, implement query budgets

Monitoring Stack:

...

Metrics Collection:





Visualization: Grafana dashboards

Alerting: PagerDuty/Slack **for** anomalies

DELIVERABLE 5: Project Roadmap & Task Breakdown

Weeks 1-2: Foundation & Setup

Research & Planning:

-  Finalize technical stack
-  Set up development environment
-  Create project GitHub repository with template
-  Design system architecture diagrams

Data Acquisition:

-  Collect 500-1000 documents in target domain (e.g., ML research papers from ArXiv)
-  Build document ingestion scripts
-  Implement chunking and preprocessing pipeline

Infrastructure:

-  Docker Compose setup for local development
-  Set up pre-commit hooks, linting, CI basics

Resume Bullet: "Designed and architected production-grade RAG system with hybrid retrieval, processing 1000+ documents into vector database"

Weeks 3-4: RAG Pipeline Development

Engineering Tasks:

-  Implement embedding generation with BGE model
-  Set up vector database (ChromaDB/FAISS)
-  Build hybrid retrieval (vector + BM25)
-  Implement reranking layer
-  Create evaluation framework for retrieval

Testing:

-  Unit tests for each component
-  Benchmark retrieval latency and accuracy
-  Create golden dataset for retrieval evaluation

Resume Bullet: "Implemented hybrid retrieval system combining semantic search and BM25, achieving 0.89 NDCG@10 on domain-specific corpus"

Weeks 5-6: LLM Fine-Tuning

Data Preparation:

-  Generate synthetic Q&A pairs from corpus (use GPT-4 API)
-  Create prompt templates for RAG
-  Manually annotate 100-200 high-quality examples

Model Training:

-  Set up Hugging Face training pipeline with QLoRA
-  Fine-tune Mistral-7B on prepared dataset
-  Implement evaluation loop (validation loss, ROUGE)
-  Hyperparameter tuning

Evaluation:

-  Compare base vs fine-tuned model
-  Human evaluation on held-out set
-  Ablation studies (with/without RAG)

Resume Bullet: "Fine-tuned Mistral-7B using QLoRA on domain-specific corpus, improving answer relevance by 35% while reducing hallucinations by 40%"

Weeks 7-8: API & UI Development

Backend:

-  Build FastAPI endpoints (query, ingestion, health)
-  Implement authentication (API keys)
-  Add request validation and error handling
-  Create comprehensive API documentation (Swagger/OpenAPI)

Frontend:

-  Build React chat interface
-  Implement streaming responses
-  Add citation highlighting and source links
-  Query history and feedback collection

Resume Bullet: "Developed RESTful API with FastAPI and React UI, handling real-time streaming responses with sub-3-second p95 latency"

Weeks 9-10: Deployment & MLOps

Cloud Deployment:

-  Set up AWS infrastructure (EC2, S3, Load Balancer)
-  Containerize all services
-  Configure CI/CD pipeline with GitHub Actions
-  Deploy staging environment

Monitoring:

-  Set up Prometheus + Grafana
-  Create dashboards for key metrics
-  Implement alerting rules
-  Add logging (structured logs to CloudWatch)

Testing:

-  Load testing (locust/k6)
-  Security scanning
-  End-to-end testing

Resume Bullet: "Deployed production MLOps pipeline with Docker, CI/CD automation, and real-time monitoring achieving 99.7% uptime"

Weeks 11-12: Optimization & Documentation

Performance Optimization:

-  Implement caching layer (Redis)
-  Model quantization (int8 inference)
-  Optimize batch sizes and async processing
-  Cost analysis and optimization

Documentation:

-  Write comprehensive README with architecture diagrams
-  Create demo video (3-5 minutes)
-  Write technical blog post explaining design decisions
-  Prepare presentation slides for interviews

Polish:

-  Code cleanup and refactoring
-  Add extensive comments
-  Create usage examples and tutorials

Resume Bullet: "Optimized inference pipeline reducing cost per query by 60% through quantization, caching, and batch processing while maintaining quality"

Stretch Goals (Optional)

Advanced Features:

- Multi-turn conversational memory
- Query understanding and reformulation
- Automatic document update detection and reindexing
- A/B testing framework for model improvements
- Federated search across multiple vector DBs

Kubernetes Deployment:

- Helm charts for reproducible deployment
- Horizontal autoscaling based on load
- Blue-green deployment strategy

Advanced MLOps:

- Drift detection for embeddings
- Automated retraining pipeline
- Feature store integration

- Model registry (MLflow)